Методика раздельного обучения трёхконтурной архитектуры для автономных систем

Автор: Максим Тимошенко
Дата: 27.04.2026
Версия: 1.1 (расширенная)

Оглавление

1. Общие принципы
2. Обучение V_h (витальный контур)
3. Обучение V_e (эпистемический контур)
4. Обучение V_s (социальный контур)
5. Обучение арбитра
6. Интеграционное тестирование
7. Вычислительные требования
8. Заключение

---

1. Общие принципы

1.1. Раздельное обучение

Три контура — V_h, V_e, V_s — обучаются **по отдельности**, каждый на своих данных и со своей функцией оценки. Это фундаментальное отличие от классических методов, где все цели закладываются в одну функцию потерь. Раздельное обучение гарантирует, что каждый контур будет экспертом в своей области, а не «компромиссным середнячком».

1.2. Порядок обучения

1. **Сначала V_h** (безопасность). Пока система не научится гарантированно предотвращать столкновения, обучать навигации бессмысленно.
2. **Затем V_e** (навигация). Когда безопасность обеспечена, система учится ездить по маршруту.
3. **Потом V_s** (социальность). Когда базовая езда освоена, добавляется взаимодействие с оператором и другими участниками движения.
4. **В конце — арбитр.** Когда все три контура работают, настраивается механизм их конкуренции.

1.3. Данные

Контур	Тип данных	Объём	Метод обучения
V_h	Критические инциденты	Десятки-сотни примеров	One-shot learning
V_e	Нормальное движение	Тысячи-десятки тысяч записей	Supervised learning

Контур	Тип данных	Объём	Метод обучения
V_s	Социальные взаимодействия	Сотни примеров	Few-shot learning
Арбитр	Симуляции конфликтов	Сотни прогонов	Эволюционная оптимизация

1.4. Калибровка

Все пороги ($\Delta = 15$, критический порог = 90, пороги TTC, температурные пороги) подбираются на симуляции **до** выезда на реальную дорогу. Это императив: ни один параметр не настраивается «в бою».

---

2. Обучение V_h (витальный контур)

2.1. Цель

Научить контур безошибочно детектировать угрозы и не давать ложных срабатываний.

2.2. Данные

Источник	Что записывается	Объём
Сенсоры безопасности (радар, лидар, камера)	Расстояние до объекта, относительная скорость, класс объекта	Десятки критических инцидентов
Датчики состояния системы	Температура, давление, уровень энергии, коды ошибок	Десятки критических инцидентов + фоновые данные

Пример для автомобиля: передний радар фиксирует расстояние и скорость сближения, передний лидар строит облако точек, передняя камера классифицирует объекты (пешеход / автомобиль / знак), CAN-шина передаёт температуру двигателя, давление масла, уровень топлива.

2.3. Метод: One-shot learning

Витальный контур не «тренируется» в классическом смысле.
Он **запоминает** критические паттерны с первого предъявления. Каждый такой паттерн — это не «веса нейросети», а правило с жёстко заданной реакцией.

Пример для автомобиля: автомобиль на скорости 90 км/ч сталкивается с внезапно выбежавшим пешеходом. TTC = 0.3 сек, расстояние = 5 м, класс объекта = «пешеход». Этот паттерн закрепляется в V_h с максимальным весом. При следующем появлении пешехода на том же расстоянии и с той же скоростью сближения V_h мгновенно выставит приоритет 95 и перехватит управление.

2.4. Код

c

```c
typedef struct {
    float ttc;
    float distance;
    float relative_speed;
    object_class_t obj_class;
    float priority;
} critical_event_t;


void vh_learn_critical_event(critical_event_t event) {
    rule_t rule;
    rule.condition.ttc_max = event.ttc;
    rule.condition.distance_max = event.distance;
    rule.condition.obj_class = event.obj_class;
    rule.action.priority = event.priority;
    vh_rule_base_add(rule);
}
```

2.5. Калибровка порогов

На симуляции прогоняются сотни сценариев с разными значениями ключевого параметра угрозы. Для каждого фиксируется: успела ли система предотвратить инцидент? Было ли ложное срабатывание?

Цель: найти такие пороги, при которых пропуск угрозы и ложное срабатывание равны нулю.

Пример для автомобиля: калибровка порогов TTC (время до столкновения). TTC > 4.0 — приоритет 30, нет перехвата. TTC < 2.0 — приоритет 60, упреждающее торможение. TTC < 1.0 — приоритет 85, интенсивное торможение. TTC < 0.5 — приоритет 100, безусловный перехват.

2.6. Калибровка порогов состояния системы

Параметр	Норма	Предупреждение	Критический порог
Параметр 1	Норма	Порог 1 (P=25)	Порог 2 (P=80)
Параметр 2	Норма	Порог 1 (P=25)	Порог 2 (P=85)
Параметр 3	100%	10% (P=40)	5% (P=70)

Пример для автомобиля: температура двигателя: норма 90°C, предупреждение 105°C (P=25), критический 120°C (P=80). Давление масла: норма 2–4 bar, предупреждение 1.5 bar (P=25), критический 0.5 bar (P=85). Топливо: норма 100%, предупреждение 10% (P=40), критический 5% (P=70).

2.7. Тестирование на ложные срабатывания

После обучения V_h прогоняется через длительную симуляцию нормальной работы. За это время не должно произойти ни одного ложного прерывания (P_vh >= критический порог без реальной угрозы). Если ложное срабатывание происходит — порог повышается или фильтр сенсора перенастраивается.

Пример для автомобиля: 1000 часов нормального движения. Ноль ложных прерываний.

2.8. Метрики успеха

Метрика	Целевое значение
Пропуск критического события	0%
Ложное прерывание	0 на 1000 часов
Время реакции при критическом событии	< 10 мс

---

3. Обучение V_e (эпистемический контур)

3.1. Цель

Научить контур следовать маршруту, соблюдать правила среды, выбирать оптимальные траектории и адаптироваться к нарушениям инфраструктуры.

3.2. Данные

Источник	Что записывается	Объём
Карта среды	Граф путей, правила, ограничения	Полный регион
Навигационные сенсоры	Траектория, скорость, курс	Тысячи часов нормаль движения
Камеры	Распознавание элементов среды	Тысячи часов
Данные о внешних условиях	Загруженность, погода	Обновляются ежеднев

Пример для автомобиля: HD-карта с дорожным графом, знаками, светофорами. GPS/ГЛОНАСС + IMU + одометры — траектория, скорость. Камеры — разметка, знаки, светофоры. Данные о трафике — скорость потока, заторы.

3.3. Метод: Supervised learning с демонстрациями эксперта

V_e учится на траекториях, записанных опытными операторами. Системе предъявляется ситуация и эталонное действие эксперта. V_e учится предсказывать это действие.

Пример для автомобиля: эксперт едет по трассе со скоростью 110 км/ч. Впереди — знак ограничения 90 км/ч. Эксперт плавно снижает скорость до 90 км/ч за 200 м до знака. V_e запоминает: «знак ограничения + расстояние → начать снижение скорости».

3.4. Код

python

```
class VELearner(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(10, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

model = VELearner()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.MSELoss()

for epoch in range(100):
```

```
for batch in expert_data:
    inputs, target = batch[0], batch[1]
    output = model(inputs)
    loss = loss_fn(output, target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

3.5. Обучение прогнозированию

V_e должен предсказывать действия других агентов в среде на несколько шагов вперёд. Для этого используется рекуррентная нейросеть (RNN), которая анализирует последовательность состояний агента за последние N секунд и предсказывает его состояние через T секунд.

Пример для автомобиля: автомобиль в соседней полосе включает правый поворотник. V_e, проанализировав его траекторию за последние 3 секунды, предсказывает: с вероятностью 80% он перестроится в нашу полосу через 3 секунды. V_e заранее снижает скорость.

3.6. Обучение поведению при нарушениях инфраструктуры

На специальных симуляциях V_e учится действовать при нарушениях ключевых элементов среды. Это не требует переобучения — это смена источника данных или логики в реальном времени.

Пример для автомобиля:

- **Отсутствие разметки:** V_e переключается на дорожный граф из HD-карты, оценивает край проезжей части через лидар, отслеживает траектории других машин.
- **Неработающий светофор:** V_e переключает логику перекрёстка на нерегулируемый режим и ищет знаки приоритета. Если знаков нет — применяет правило «помеха справа».

3.7. Метрики успеха

Метрика	Целевое значение
Отклонение от маршрута	< 50 м на 300 км
Успешный проезд узлов без ключевых элементов инфраструктуры	100%
Точность прогнозирования действий других агентов	> 90%

---

4. Обучение V_s (социальный контур)

4.1. Цель

Научить контур взаимодействовать с оператором, другими агентами и специальными службами.

4.2. Данные

Источник	Что записывается	Объём
Приёмник голосовых команд	Команды оператора	Сотни записей
Камера	Жесты оператора и регулировщика	Десятки-сотни записей
Внешние микрофоны	Сигналы специальных служб	Десятки записей
Модем связи	Сообщения от других агентов и инфраструктуры	Сотни сообщений

Пример для автомобиля: микрофон в салоне — голосовые команды водителя. Внешние камеры — жесты регулировщика, невербальные сигналы

других водителей. Внешние микрофоны — сирены спецтранспорта. V2X-модем — сообщения от других ТС.

4.3. Метод: Few-shot learning

V_s должен обобщать с малого количества примеров. Если он увидел три разных образца одного класса, он должен научиться распознавать четвёртый, не встречавшийся ранее.

Пример для автомобиля: V_s предъявляются видеозаписи трёх жестов регулировщика (STOP, прямо, направо). После few-shot обучения он должен распознать четвёртый жест (круговое движение жезлом), даже если он не встречался в обучающей выборке.

4.4. Код

python

```
import torchvision.models as models


base_model = models.resnet18(pretrained=True)

num_classes = 4

base_model.fc = nn.Linear(base_model.fc.in_features, num_classes)


few_shot_data = load_officer_gestures()


for epoch in range(50):

    for images, labels in few_shot_data:

        output = base_model(images)

        loss = loss_fn(output, labels)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()
```

4.5. Обучение градации приоритетов

Самый важный этап обучения V_s — правильное назначение приоритета командам. Команды оператора не исполняются слепо — они проходят через арбитраж.

Пример для автомобиля:

Команда	Приоритет	Реакция системы
«Поехали быстрее»	25	V_e учитывает пожелание, но не превышает ограничение
«Перестройся левее»	30	V_s перехватывает управление у V_e, если полоса свободна
«Останови здесь!» (на магистрали)	55	V_s перехватывает управление, но предохранитель блокирует остановку
«Стой, опасно!!!»	65	V_s перехватывает управление немедленно, экстренное торможение

Критически важно: приоритет команды регулировщика (70) должен быть ниже критического порога V_h (90). Это гарантирует, что даже регулировщик не заставит систему совершить опасное действие.

4.6. Обучение невербальной коммуникации

V_s должен освоить стандартные сигналы, используемые в среде. Это жёстко запрограммированные действия, не требующие машинного обучения.

Пример для автомобиля:

Сигнал	Когда подавать	Что означает
Короткое мигание дальним (0.3 сек)	При пропуске другого автомобиля	«Проезжай, уступаю»
Двойное мигание аварийкой (1 сек)	После того, как вас пропустили	«Спасибо»

Сигнал	Когда подавать	Что означает
Указатель поворота за 5 сек до манёвра	Перед перестроением или поворотом	«Планирую манёвр»
Короткий гудок (0.2 сек)	При обнаружении опасности	«Осторожно!»

4.7. Метрики успеха

Метрика	Целевое значение
Распознавание голосовых команд оператора	> 95%
Распознавание жестов регулировщика	100%
Правильная градация приоритетов команд	100%
Распознавание сигналов специальных служб	100%

---

5. Обучение арбитра

5.1. Цель

Настроить механизм конкуренции так, чтобы система не «дребезжала» и не зависала в критических ситуациях.

5.2. Настраиваемые параметры

Параметр	Предварительное значение	Что оптимизируется
Δ (гистерезис)	15	Предотвращение «дребезга»
Критический порог V_h	90	Гарантированное время реакции

5.3. Метод: эволюционная оптимизация

Арбитр настраивается на сотнях симуляций конфликтных сценариев. Целевая функция: **0 пропущенных угроз + 0 ложных срабатываний + 0 «дребезгов»**.

Пример для автомобиля: симулируется сценарий «регулировщик приказывает ехать на красный, но выбегает пешеход». На каждой итерации меняются Δ и критический порог. Отбираются те значения, при которых система успела затормозить перед пешеходом и не переключалась туда-сюда между V_s и V_h более одного раза за секунду.

5.4. Код

python

```
import random


def run_simulation(delta, critical):

    score = 0

    for scenario in SCENARIOS:

        result = simulate_scenario(scenario, delta, critical)

        if result.passed:          score += 1

        if result.false_positive:  score -= 10

        if result.missed_threat:   score -= 100

        if result.oscillations > 0: score -= 5

    return score


population = [(random.uniform(10, 20), random.uniform(85, 95)) for _ in
range(50)]


for generation in range(100):
```

```
    scores = [(run_simulation(d, c), d, c) for d, c in population]
    scores.sort(reverse=True)
    best = scores[:10]
    new_population = []
    for _ in range(50):
        parent = random.choice(best)
        child_delta = parent[1] + random.uniform(-1, 1)
        child_critical = parent[2] + random.uniform(-1, 1)
        new_population.append((child_delta, child_critical))
    population = new_population


best_score, best_delta, best_critical = scores[0]
print(f"Оптимальные параметры: Δ = {best_delta:.1f}, критический порог = {best_critical:.1f}")
```

5.5. Результат калибровки

После оптимизации параметры могут немного отличаться от исходных (например, Δ = 14, критический порог = 88), обеспечивая лучший баланс между быстродействием и стабильностью.

5.6. Тестирование арбитра

После настройки арбитр прогоняется через все обязательные сценарии (см. раздел 6). Если хотя бы в одном сценарии система не показала ожидаемое поведение — параметры возвращаются на доработку.

5.7. Метрики успеха

Метрика	Целевое значение
Время переключения на V_h при критическом событии	< 10 мс

Метрика	Целевое значение
Количество «дребезгов» (переключений туда-сюда за 1 сек)	0
Успешное прохождение всех обязательных сценариев	100%

---

6. Интеграционное тестирование

После обучения всех контуров и арбитра система проходит обязательный набор сценариев. Каждый сценарий прогоняется не менее 100 раз в различных вариациях.

№	Сценарий	Ожидаемое поведение	Кто побеждает
1	Нормальное движение	Агент следует маршруту, держит параметры	V_e
2	Внезапная критическая угроза	Экстренная остановка, сигнал	V_h (безусловно)
3	Регулировщик приказывает нарушить стандартное правило	Агент исполняет команду	V_s
4	Регулировщик приказывает, но возникает критическая угроза	Экстренная остановка, приказ регулировщика игнорируется	V_h (безусловно)
5	Отказ ключевого элемента инфраструктуры	Переключение на альтернативную логику	V_e

№	Сценарий	Ожидаемое поведение	Кто побеждает
6	Отсутствие ключевого элемента среды	Позиционирование по альтернативным источникам	V_e
7	Оператор требует нарушить правила безопасности	Команда заблокирована предохранителем	Предохранитель
8	Специальная служба требует уступить	Агент уступает	V_s

Пример для автомобиля: сценарии 1–8 из раздела 6 оригинальной методики (нормальное движение, пешеход, регулировщик, регулировщик + пешеход, отказ светофора, отсутствие разметки, остановка на магистрали, спецтранспорт).

Критерий успеха: все сценарии × 100 прогонов. Допустимое количество отклонений — **0**.

---

7. Вычислительные требования

Компонент	Требование
GPU	1 × NVIDIA RTX 4090 или аналог (24 ГБ VRAM)
CPU	16–32 ядра
RAM	64 ГБ
Накопитель	2 ТБ SSD

Примерное время обучения:

Этап	Время
V_h (one-shot)	Минуты

Этап	Время
V_e (supervised)	Часы
V_s (few-shot)	Часы
Арбитр (эволюционная оптимизация)	Часы
Интеграционное тестирование	Часы
Итого	**1–2 рабочих дня**

---

8. Заключение

Предложенная методика описывает полный цикл обучения трёхконтурной архитектуры — от one-shot обучения безопасности до эволюционной оптимизации арбитра. Ключевые преимущества:

1. **Раздельное обучение** — каждый контур обучается независимо.
2. **Фиксированный порядок** — безопасность всегда первая.
3. **Калибровка на симуляции** — ни один параметр не настраивается «в поле».
4. **Низкие вычислительные требования** — обучение на одной рабочей станции за 1–2 дня.

Методика предназначена для ML-инженеров и разработчиков автономных систем, реализующих трёхконтурную архитектуру на практике.